

Implementation of a Secure DevOps CI/CD Pipeline Using DevSecOps Tools

Revanth Kumar T S

Department of Computer Science and Engineering
Global Academy of Technology
Bengaluru, India
revanthkumarts123@gmail.com

Sanket

Department of Computer Science and Engineering
Global Academy of Technology
Bengaluru, India
sankethcoding@gmail.com

Vignesh P

Department of Computer Science and Engineering
Global Academy of Technology
Bengaluru, India
palanivignesh31@gmail.com

Shakthi Vel K

Department of Computer Science and Engineering
Global Academy of Technology
Bengaluru, India
shakthivelk1124@gmail.com

Sameena H S

Department of Computer Science and Engineering
Global Academy of Technology
Bengaluru, India
sameena.hs@gat.ac.in

Abstract— DevOps is followed in many projects mainly to make the work faster by keeping development and operations more connected. With increasing security needs, DevSecOps has come into use, where security is considered from the early stages instead of adding it later.

In practice, some challenges still exist, such as difficulty in automating security checks, poor coordination between tools, limited security knowledge among developers, and incorrect alerts from scanning tools.

In this work, a framework is presented to improve security checking in the DevSecOps pipeline. In our setup, tools like Git, Jenkins, Docker, Kubernetes and SonarQube are used. For security checking, OWASP ZAP and Trivy are included, and for monitoring we used Prometheus with Grafana.

The framework is kept in between CI and CD. Because of this, checking happens while the code is still moving in the pipeline. If something goes wrong, it can be noticed earlier itself, so later fixing work will be less. A simple design and one case example are also shown.

Keywords— DevOps, DevSecOps, CI/CD pipeline, security testing, vulnerability scanning, containerization, monitoring tools, automated security, software security

I. INTRODUCTION

As many organizations move towards DevOps and agile practices, CI/CD pipelines have become a regular part of software development. In our setup, a container-based workflow is used to keep the application behaviour same across different environments. It also helps in making deployment a bit easier to manage. At the same time, security is becoming more important, so DevSecOps is followed where security is taken care of from the beginning stages itself instead of adding it later.

Security checking is an important part of this process, especially to detect issues before deployment. Earlier, these checks required more manual effort and often caused delays. Here, the idea is to automate security-related checks so that issues can be found faster during development, without waiting until the end.

For this, both code-level and runtime checking are used. For checking the code, SonarQube is used. At the same time,

OWASP ZAP and Trivy are used to look for issues when the application is running or inside the container. Jenkins is used to handle the pipeline, and Docker along with Kubernetes is used for managing the containers.

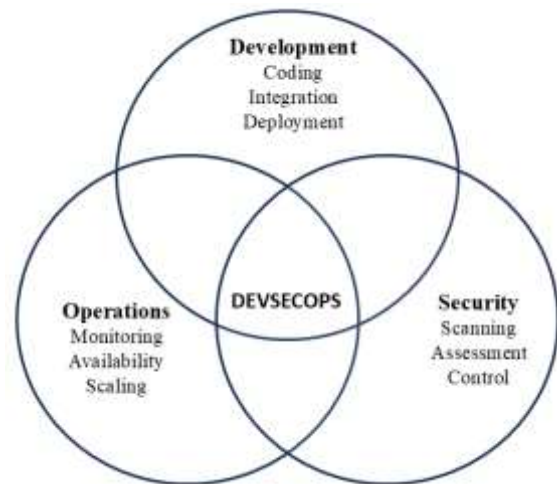


Fig. 1. DevSecOps

For monitoring, Prometheus and Grafana are used. With this, the system can be watched continuously, and if any problem happens, it can be noticed.

Overall, this setup helps in finding problems earlier. Because of that, fixing becomes easier and it does not delay the development work much.

II. LITERATURE SURVEY

Early work in DevSecOps mainly tried to understand why security was often missed in DevOps pipelines. For example, Nikolov and Aleksieva-Petrova (2023) worked on a Jenkins-based pipeline where tools like Snyk and OWASP ZAP were used for security checking. In their case, the system was able to find vulnerabilities before deployment, but they did not include container-level security or any monitoring part.

In another work, Marandi et al. In one work from 2023, Snyk was used along with StackHawk in a GitHub Actions pipeline with Docker. They also added dashboards and some level of automation for fixing issues, so manual work was

reduced. But still, container scanning and runtime monitoring were not really included in that setup.

Later, more attention was given to the problems faced while actually applying DevSecOps. Nayanajith and Wickramarachchi (2024) mentioned that issues like lack of skills, resistance from teams, and communication gaps were quite common. These were not mainly technical problems, but more about how teams coordinate and work together in practice.

Bautista Ramos and Yoo (2025) looked at many research papers and discussed different kinds of vulnerabilities along with possible ways to handle them. They also mentioned that a lot of these studies were not tested in real environments, so it is not very clear how they would work in actual use.

Some recent work has focused more on practical pipeline setups. In one such case, tools like Jenkins, Docker, and Kubernetes were used together with Prometheus and Grafana to improve both system performance and monitoring in CI/CD pipelines. It was also noted that Trivy can be used to check container-related issues.

There is also work based on the CAMS model in DevSecOps, which includes culture, automation, measurement, and sharing. These factors are considered important when applying DevSecOps in real situations. These aspects are considered important when trying to apply DevSecOps in practical situations. These factors are important when applying DevSecOps in real environments. It showed that these factors play a key role in successfully applying DevSecOps in real environments.

In addition, recent research has explored improving the monitoring phase using machine learning. A hybrid intrusion detection system was proposed to enhance security during continuous monitoring, achieving high accuracy in detecting threats. However, such approaches are still limited to specific datasets and need more real-world validation.

From these studies, it can be understood that DevSecOps is improving, but some gaps still remain. From these studies, it can be seen that many works focus only on specific parts such as scanning or monitoring, but do not cover the complete pipeline.

In this work, a complete DevSecOps pipeline is implemented where security is included at every stage. From these works, it can be understood that most studies focus only on some parts of the pipeline. In this work, the idea is to combine different stages together. It includes static and dynamic testing, container scanning using Trivy, and monitoring using Prometheus and Grafana so that the system can be checked continuously.

Because of this, there is a need to combine different stages of the pipeline into a single system. In this work, an attempt is made to bring these parts together so that security can be checked continuously at each stage.

III. PRINCIPLES OF SECURE DEVOPS

In many cases, organizations face problems when trying to use DevSecOps in actual projects. One common issue is that security is usually taken care of at the final stage, which later creates extra work. At the same time, development teams try to move faster, but if security is not handled properly, it may slow things down. Because of this, managing both speed and security together becomes a bit difficult.

Another issue is that not everyone in the team has proper understanding of security practices. Developers may give less attention to security, and security teams may not fully know the development side. This creates a gap between teams, which can affect the system.

In our work, we tried to avoid these problems by adding security checks directly into the pipeline so that they run along with development steps.

A. Principles Used in Our Work

1) Security from Early Stage

Instead of adding security later, checking is done from the beginning. Code is checked during development itself so that issues can be found early.

2) Automation in Pipeline

Most of the steps are automated. When code is pushed, the pipeline starts and runs the required steps like checking and building without much manual work.

3) Continuous Process

The system works in a continuous manner where the flow is not interrupted. Code changes move step by step without stopping, which helps in faster development.

4) Different Levels of Checking

Security is checked in more than one way. Checking is done in stages. First the code is checked, then the application, and later the running system is verified.

5) Container Checking

Before deployment, the container is also checked so that any major issue can be noticed at that stage itself.

6) Monitoring After Deployment

After deployment, the system is still observed regularly. This helps in identifying if any issue comes later.

7) Team Involvement

All stages are connected, so work is not limited to one team. This makes it a bit easier for teams to follow the process and work together without too much confusion.

8) Managed Setup

Deployment is handled in a controlled way so that the system stays stable and is easier to manage afterwards.

Using this approach, the system becomes more secure and simpler to manage. Issues can be noticed earlier, and overall the process becomes easier to deal with.

IV. DEVSECOPS PIPELINE MODEL

The pipeline used in this work is shown in Fig. 1. In our case, development and security are handled together as the process moves step by step, instead of doing them separately. The idea is to check issues at different stages, not only at the end.



Fig. 2. Proposed DevSecOps pipeline model

The process starts when code is taken from the Git repository. After that, the code is checked using SonarQube so that basic issues can be found early. After this, the application is built using tools like Node.js, based on what is used in the project.

Next, a Docker image is created. This image is then checked using Trivy to see if there are any problems inside the container. Only after this, the application is moved to deployment using Kubernetes.

After deployment, the running application is tested using OWASP ZAP. This helps in finding issues that may not be visible during earlier stages.

At the final stage, monitoring is done using Prometheus and Grafana. This helps in keeping track of the system regularly, so if any issue comes later, it can be noticed.

Overall, the pipeline keeps running step by step, and checking is done along with the development process. If any issue comes, it can be noticed at an earlier stage, so fixing it later does not become difficult.

V. METHODOLOGY

In this work, a DevSecOps pipeline is implemented where different stages are connected in a continuous flow. The main aim is to include security checks along with development instead of handling them separately.

The process begins when the code is pushed to the Git repository. After that, the pipeline gets triggered and the code

is taken for further steps. In the next stage, static checking is done to find basic issues in the code.

Once this is completed, the application is built. After building, a container image is created using Docker. This image is then checked to identify any vulnerabilities present inside it.

After the checking is done, the application is deployed using Kubernetes. After the application starts running, it is tested to check if any issues are present during execution.

At the last stage, the system is observed regularly to see if any problem occurs. Tools are used to observe system performance and detect problems during runtime. The details of these tools are shown in Table I.

Sl. No	Tool	Category	Purpose
1	Git	Version Control	Track code changes
2	Jenkins	CI/CD Tool	Automate build and deploy
3	Docker	Containerization	Package application
4	Kubernetes	Container Orchestration	Manage containers
5	SonarQube	SAST Tool	Detect code issues

Table I: Tools Used in DevSecOps Pipeline

In this way, all stages are connected, and security checking is done at multiple points. This helps in identifying issues earlier and reduces the effort required at later stages.

VI. RESULTS AND DISCUSSION

In this work, the DevSecOps pipeline was implemented and tested to check how it performs in different stages. The pipeline was able to run continuously from code integration to deployment without major interruption.

In the early stages, checking the code helped to find issues sooner, so fewer problems appeared later. Container checking also helped in finding problems before deployment, so only verified images were used.

After deployment, the application was checked while it was running. This made it possible to notice some issues that were not seen in the earlier stages. Monitoring was also carried out to observe system behaviour, and it helped in tracking performance and detecting any issues during runtime.

One thing we observed is that when checking is done at different stages, fewer problems remain at the end. Because of this, less effort is needed to fix things after deployment.

Overall, combining development and security in the same flow made the process better. This also made the system more stable and easier to manage compared to handling security as a separate step.

VII. COMPARISON WITH EXISTING WORK

When compared with earlier works, most of them focused only on specific parts of the pipeline such as code checking or testing after deployment. In some cases, tools were used only for static analysis or only for dynamic testing. Because of this, security was not covered at all stages.

In our case, the pipeline includes checking at different points, such as code level, container level, and after deployment. This makes the process more continuous instead of handling security in a single step.

Another difference is that some existing works did not include proper monitoring after deployment. In this work, monitoring is also included, so the system can be observed even after it is running.

From this, it can be understood that combining multiple stages together helps in improving the overall process and reduces issues at later stages. The comparison between existing work and the proposed system is shown in Table II.

Feature	Existing Work	Proposed Work
Code Checking	Yes	Yes
Dynamic Testing	Partial	Yes
Container Scan	No	Yes
Monitoring	Limited	Yes
Full Pipeline	No	Yes

Table II: Comparison with Existing Work

From the table, it can be seen that the proposed work covers more stages compared to earlier approaches.

VIII.CONCLUSION

In this work, a DevSecOps pipeline is proposed by combining development and security in a single flow. The main idea is to include checking at different stages instead of handling security only at the end.

Based on the design, early checking can help in finding issues sooner, which may reduce problems in later stages. Checking the container and also testing after deployment can help in finding some issues that are not seen in the early stages. Monitoring can also be used to keep an eye on the system even after it is deployed.

The pipeline is planned in such a way that it runs step by step, with all stages connected. Because of this, the process can be easier to handle and may reduce extra work later.

In future, this can be improved by adding more features and making the automation better so that the system works in a smoother way.

REFERENCES

- [1] M. Marandi, A. Bertia, and S. Silas, "Implementing and automating security scanning in a DevSecOps CI/CD pipeline," in *Proc. World Conf. Communication & Computing (WCONF)*, Raipur, India, Jul. 2023.
- [2] M. Voggenreiter, F. Angermeir, F. Moyón, U. Schöpp, and P. Bonvin, "Automated security findings management: A case study in industrial DevOps," in *Proc. 46th Int. Conf. Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, Apr. 2024.
- [3] A. Nayanajith and R. Wickramarachchi, "Challenges affecting the successful adoption of DevOps practices: A systematic literature review," in *Proc. 4th Int. Conf. Advanced Research in Computing (ICARC)*, 2024.
- [4] N. Dinh *et al.*, "Enhancing smart contract security through DevSecOps: An adaptive approach for vulnerability detection," *IEEE Access*, vol. 13, pp. 159454–159520, 2025.
- [5] R. Ashtagi *et al.*, "Building resilient CI/CD pipelines: A DevOps security-first framework," in *Proc. Int. Conf. Computational, Communication and Information Technology (ICCCIT)*, 2025.
- [6] R. C. B. Ramos and S. G. Yoo, "Cybersecurity in DevOps environments: A systematic literature review," *IEEE Access*, vol. 13, pp. 191959–191980, 2025.
- [7] S. A. Ruk *et al.*, "Effort estimation in Agile and DevOps: A systematic literature review of stakeholder dissatisfaction," *IEEE Access*, vol. 13, pp. 167925–167960, 2025.
- [8] N. S. P. Manja, P. Mathur, and P. B. Honnavalli, "Extensive review of threat models for DevSecOps," *IEEE Access*, vol. 13, pp. 45252–45274, 2025.
- [9] M. A. Akbar and A. A. Alsanad, "Empirical investigation of key enablers for secure DevOps practices," *IEEE Access*, vol. 13, pp. 43698–43710, 2025.
- [10] J. Ababneh *et al.*, "Enhancing DevOps continuous monitoring phase: Hybrid intrusion detection and ensemble learning system," *IEEE Access*, vol. 14, pp. 4733–4755, 2026.